

TP 2 : Fonctions et structures de données

Ali ZAINOUL
ali.zainoul.az@gmail.com

November 9, 2025

Objectifs pédagogiques :

- Comprendre les différents types de fonctions MATLAB (avec ou sans entrées et sorties).
- S'exercer à créer des fonctions manipulant les structures de données courantes : vecteurs, matrices, tableaux de cellules et tableaux de structures.
- Développer des habitudes de test en écrivant de petits scripts pour valider le comportement des fonctions.

Consignes : Pour chaque section ci-dessous, lisez attentivement l'explication, examinez le code d'exemple, puis complétez les exercices. Utilisez des commentaires dans vos fonctions pour expliquer votre logique.

1. Types de fonctions en MATLAB

Les fonctions MATLAB diffèrent selon qu'elles acceptent ou non des entrées et retournent ou non des sorties. Choisir la bonne signature de fonction aide à écrire un code clair et modulaire.

Type	Entrées	Sorties
Procédure	0	0
Action	≥ 1	0
Générateur	0	≥ 1
Transformateur	≥ 1	≥ 1

Table 1: Résumé des catégories de fonctions MATLAB

1.1 Procédure : Aucune entrée, aucune sortie

But : Effectuer une tâche (affichage, initialisation, etc.) sans retourner de données.

Exemple :

```
function greet()
% Display a welcome message to the user
disp('Welcome to this MATLAB workshop!');
end
```

Conseil : Utilisez ces fonctions pour regrouper du code d’affichage ou d’initialisation qui ne nécessite pas de données de retour.

1.2 Action : Entrée(s), aucune sortie

But : Exécuter une action à partir d’une donnée d’entrée (par ex. afficher, tracer) sans retourner de valeur.

Exemple :

```
function print_vector(v)
% Display each element of vector v with its index
for k = 1:length(v)
    fprintf('v(%d) = %g\n', k, v(k));
end
end
```

Astuce : Considérez `v` comme en lecture seule ; toute modification ne sortira pas de la fonction.

1.3 Générateur : Aucune entrée, une sortie

But : Créer et renvoyer des données sans arguments d’entrée.

Exemple :

```
function m = random_matrix()
% Return a 3x3 matrix of random values uniformly sampled in [0,1]
m = rand(3);
end
```

Vérification : Appelez `m = random_matrix()` ; puis inspectez `size(m)` et `min(m(:))`.

1.4 Transformateur simple : Une entrée, une sortie

But : Calculer et renvoyer un seul résultat dérivé d’une entrée.

Exemple :

```
function s = sum_vector(v)
% Return the sum of all elements in vector v
s = sum(v);
end
```

Exercice : Que se passe-t-il si `v` est une matrice 2D ? Essayez et documentez le comportement.

1.5 Transformateur général : Plusieurs entrées, plusieurs sorties

But : Réaliser des calculs complexes nécessitant plusieurs paramètres et retourner un ou plusieurs résultats.

Exemple :

```
function [mean_val, std_val] = stats_vector(v, dim)
% Compute the mean and standard deviation along dimension dim of v
mean_val = mean(v, dim);
std_val = std(v, 0, dim);
end
```

Conseil : Vérifiez toujours les dimensions : utilisez `size(v)` pour valider `dim`.

2. Exercices

Pour chaque exercice :

1. Définissez la fonction dans son propre fichier `.m`.
2. Rédigez un court script de test pour appeler la fonction avec des entrées pertinentes.
3. Ajoutez des commentaires expliquant vos choix d'implémentation.

Exercice 1 : Fonction de type procédure (vecteur)

Objectif : Créer et afficher un vecteur fixe.

- Nom de la fonction : `display_unit_vector`
- Tâche : Générer le vecteur colonne `[1;0;0;0;0]` et l'afficher.

Script de test :

```
display_unit_vector(); % expect [1;0;0;0;0]
```

Exercice 2 : Fonction d'action (matrice)

Objectif : Afficher les propriétés d'une matrice.

- Nom de la fonction : `show_matrix_stats(m)`
- Tâche : Afficher le texte "Size: nxm, Elements: p" où `n,m = size(m)` et `p = numel(m)`.

Script de test :

```
a = magic(4);
show_matrix_stats(a);
% Expected output: "Size: 4x4, Elements: 16"
```

Exercice 3 : Fonction génératrice (tableau de cellules)

Objectif : Construire et renvoyer une liste de noms.

- Nom de la fonction : `create_name_list`
- Tâche : Retourner un tableau de cellules contenant exactement trois noms.

Script de test :

```
names = create_name_list();  
disp(names);
```

Exercice 4 : Transformateur simple (structure)

Objectif : Initialiser une fiche étudiant.

- Nom de la fonction : `init_student(id)`
- Tâche : Retourner une structure avec les champs :
 - `id` : valeur d'entrée
 - `grades` : `{}` (cellule vide)
 - `average` : 0

Script de test :

```
stud = init_student(42);  
assert(stud.id == 42);  
assert isempty(stud.grades);  
assert(stud.average == 0);
```

Exercice 5 : Transformateur général (vecteur)

Objectif : Mettre à l'échelle et décaler un vecteur.

- Nom de la fonction : `scale_and_translate(v, alpha, beta)`
- Tâche : Retourner deux sorties :
 - $u = \alpha * v$
 - $w = v + \beta$

Script de test :

```
v = [1,2,3];  
[u, w] = scale_and_translate(v, 2, 5);  
assert(isequal(u, [2,4,6]));  
assert(isequal(w, [6,7,8]));
```

Exercice 6 : Fonction mixte (matrice, cellules, structures)

Objectif : Résumer chaque ligne avec des métadonnées.

- Nom de la fonction : `summary_data(m, names)`
- Entrées :
 - `m` : matrice $n \times m$
 - `names` : tableau de cellules de n chaînes
- Sortie : `s` (tableau de structures $1 \times n$) avec les champs :
 - `name` : `names{i}`
 - `sum_row` : `sum(m(i,:))`
 - `max_row` : `max(m(i,:))`

Script de test :

```
names = {'Lyon', 'Paris', 'Marseille'};
m = [1 4 2; 3 5 7; 6 2 9];
s = summary_data(m, names);
assert(strcmp(s(2).name, 'Paris'));
assert(s(2).sum_row == 15);
assert(s(2).max_row == 7);
```